

Time-Shifted Encryption Web Platform

Submitted by:

Guntur Ridhi

Registration Number: **AP22110011467**

B.Tech, Computer Science and Engineering

Under the guidance of:

Dr. Sazzad Ali Biswas

Assistant Professor

Department of Mathematics

Semester: Jan – Apr 2025



SRM University, Andhra Pradesh

Department of Computer Science and Engineering

Contents

1	Introduction	1
1.1	Background	1
1.2	Purpose of the Project	1
2	Problem Statement	1
2.1	Real-World Motivation	1
2.2	Challenges Addressed	1
3	Objective	2
3.1	Main Goals	2
3.2	Target Outcome	2
4	Scope of the Project	3
4.1	What the Project Covers	3
4.2	What is Not Included	4
5	Technology Stack Used	4
5.1	Frontend – React.js	4
5.2	Backend – Flask (Python)	5
6	6. System Architecture	5
6.1	High-Level Design	5
6.2	Flow Diagram	5
7	Working Methodology	6
7.1	File Upload and Time Input	7
7.2	AES Encryption Process	7
8	Modules Implemented	8
8.1	File Upload Module	8
8.2	Encryption Module	9
9	AES Encryption Used in This Project	9
9.1	Why AES?	9
9.2	Mode of Operation: AES-CBC (Cipher Block Chaining)	10
9.3	AES Encryption Rounds	10
10	User Interface (Frontend)	11
10.1	Upload Page	11
10.2	Download & Decrypt Page	11
11	Backend Workflow	11
11.1	Flask API Endpoints	11
11.2	File Handling	11

12 Time Lock Mechanism	11
12.1 Time Comparison Logic	11
12.2 Preventing Early Access	12
13 Security Considerations	12
13.1 Confidentiality	12
13.2 Integrity	12
14 Testing and Results	12
14.1 Functional Testing	12
14.2 Sample Screenshots	13
15 Limitations	14
15.1 Only Text File Supported	14
15.2 Frontend Upload Errors	14
16 Conclusion	14
16.1 Summary	14
16.2 Learnings	14
17 Future Work	14
17.1 Support for PDF, Image, and Video	14
17.2 Decryption Link Sharing	15
18 References	15
19 Appendix	15
19.1 Code Snippets	16

1. Introduction

1.1. Background

In today's digital age, the need for secure and time-sensitive communication is more critical than ever. With the exponential rise in data sharing across networks, ensuring that information is both encrypted and accessed only at the intended time has become a key requirement in many sectors, including academia, business, and personal communication. Traditional encryption techniques provide confidentiality but do not offer built-in mechanisms to restrict access based on time. This limitation led to the concept of "Time-Shifted Encryption," where encrypted content is only decryptable after a user-defined future time.

This project aims to implement such a mechanism by integrating time-based access control with standard encryption practices. The project utilizes the Advanced Encryption Standard (AES), a widely accepted symmetric encryption algorithm, combined with a timestamp verification system that controls the exact moment when a user can decrypt the content.

1.2. Purpose of the Project

The primary goal of this project is to develop a simple yet secure web platform that allows users to upload a text file, specify a future unlock time, and receive an encrypted file that cannot be decrypted before that time. Once the specified time has passed, the same platform allows the user to decrypt the file and retrieve the original content.

This system ensures that even if the encrypted file is shared or intercepted, it remains unusable until the allowed time. This adds an additional layer of security and control, particularly in scenarios like delayed content releases, timed exams, scheduled document sharing, or secret message delivery. The project emphasizes practicality, focusing on the use of standard web technologies—React.js for the frontend and Flask for the backend—while implementing the cryptographic logic and time-lock mechanism effectively.

2. Problem Statement

2.1. Real-World Motivation

In various practical situations, it becomes crucial to share sensitive files or messages that must remain inaccessible until a specific future time. For example, in educational environments where question papers are distributed in advance, or in legal and corporate setups where documents must be time-released. Additionally, individuals may wish to send personal messages or surprises that should be opened only on a particular date. Traditional encryption ensures confidentiality but doesn't restrict access based on time. This gap highlights the need for a mechanism that allows files to remain encrypted and inaccessible until a predefined unlock time.

2.2. Challenges Addressed

This project addresses the challenge of enforcing time-based access control on encrypted files. The key issues resolved include:

- Preventing premature access to sensitive data despite having the encrypted file.
- Building a user-friendly interface that accepts file input and unlock time.
- Implementing secure AES encryption that integrates with a time-check mechanism.
- Ensuring the decryption only proceeds when the system's current time exceeds the user-defined unlock time.
- Handling basic file formats (text files in this case) securely and efficiently.

3. Objective

3.1. Main Goals

The primary goal of this project is to create a Time-Shifted Encryption Web Platform that allows users to securely encrypt messages or files, with the ability to decrypt them only after a specified future time. This platform will serve as a tool to explore the concept of time-locked encryption and offer a secure, user-friendly way for users to send messages or files to others with time-based access restrictions.

Key objectives of this project include:

- **File Upload and Encryption:** Allow users to upload files (text, images, PDFs, or videos), encrypt them using AES encryption, and set a specific unlock time when the file can be decrypted.
- **Decryption:** Provide functionality to decrypt the encrypted files only after the specified unlock time has passed.
- **User Authentication:** Enable users to register, log in, and securely access the platform.
- **Time Locking Mechanism:** Implement a mechanism where files or messages are time-locked using an AES key, ensuring they cannot be accessed until the designated unlock time.
- **Intuitive User Interface:** Build an easy-to-use frontend where users can interact with the platform without requiring in-depth technical knowledge.

3.2. Target Outcome

The target outcome of this project is to successfully build a fully functional web platform with the following capabilities:

- **Secure User Authentication:** Ensure that users can securely register, log in, and perform actions like uploading and decrypting files.
- **AES-Based Encryption:** Implement AES encryption to securely lock files and ensure the integrity of the files during the encryption and decryption process.

- **Time-Locked Decryption:** Files or messages will remain encrypted and inaccessible until the designated unlock time, ensuring that the intended recipient can only access the file once the time has passed.
- **Seamless File Management:** Allow users to upload, store, and download encrypted files, as well as perform decryption after the specified unlock time.
- **Error Handling and User Feedback:** Provide clear feedback to users during file upload, encryption, and decryption processes, ensuring that they are aware of any errors or issues (e.g., attempting to decrypt a file before the unlock time).

Once the platform is fully functional, it will demonstrate a novel approach to secure communication, offering a new layer of privacy and security to users who need to send time-sensitive information. The platform aims to be useful in contexts such as legal document sharing, personal communication, and time-sensitive business interactions, where confidentiality and timing are critical.

4. Scope of the Project

4.1. What the Project Covers

The scope of this project encompasses the development of a **Time-Shifted Encryption Web Platform** that allows users to securely upload files, encrypt them using AES encryption, and set a specific unlock time. The platform will ensure that these files are only accessible after the specified time, with the following features covered:

- **File Upload and Encryption:** Users can upload various types of files (text, images, PDFs, videos) and encrypt them securely with AES encryption.
- **Time Locking Mechanism:** The files will be locked using a time-specific encryption key, ensuring that decryption can only occur once the unlock time is reached.
- **Decryption after Unlock Time:** Users will be able to decrypt the files only after the designated unlock time has passed.
- **User Authentication:** The platform will support secure user authentication via a registration and login system, ensuring that only authorized users can upload, manage, and decrypt files.
- **Seamless File Management:** Users will have the ability to upload, download, and store encrypted files on the platform, along with options to perform the decryption process once the unlock time arrives.
- **User-Friendly Interface:** A simple and intuitive user interface (UI) will be developed for the platform, enabling users to interact easily without requiring technical knowledge.
- **Error Handling:** The platform will handle errors efficiently by providing appropriate feedback to users, especially for issues such as invalid file uploads or attempting to decrypt files before the unlock time.

4.2. What is Not Included

While the project aims to provide a comprehensive solution for time-locked encryption, there are certain limitations and exclusions:

- **Mobile App Version:** The project will not include the development of a mobile app or mobile-specific features. The platform will solely focus on web-based functionality.
- **Complex File Formats:** The platform will not include support for all possible file formats. For example, highly complex or proprietary formats like database files or interactive media will not be considered for encryption.
- **Real-Time Messaging:** The project will not support real-time messaging or communication features. It will only support file encryption and decryption based on a fixed unlock time.
- **Advanced Encryption Methods:** While AES encryption will be implemented, other advanced encryption algorithms or methods (e.g., RSA, Elliptic Curve Cryptography) will not be part of the current scope.
- **Cloud Storage Integration:** The project will not include integration with third-party cloud storage providers (e.g., Google Drive, Dropbox). Files will be stored locally on the platform's server.
- **User Roles and Permissions:** The platform will not include multiple user roles or complex permission systems. It will operate on a single-user basis for file upload and decryption.
- **Support for Offline Use:** The platform will be entirely web-based and will not include support for offline access to encrypted files or offline decryption.

5. Technology Stack Used

5.1. Frontend – React.js

The frontend of the Time-Shifted Encryption Web Platform will be built using **React.js**, a widely used JavaScript library for building user interfaces, particularly for single-page applications (SPAs). React's component-based architecture allows for efficient updates and rendering, making it suitable for creating dynamic and responsive web pages. Key features and reasons for choosing React.js include:

- **Component-Based Architecture:** React allows for the development of modular, reusable components, which simplifies the UI development process and ensures maintainability.
- **Virtual DOM:** React's Virtual DOM improves performance by reducing the number of direct manipulations to the real DOM, providing a smooth user experience.
- **Fast Rendering:** React's ability to efficiently update and render only the components that have changed makes it highly performant, particularly for interactive applications.

- **Strong Ecosystem:** React has a vast ecosystem of tools, libraries, and community support, which enables rapid development and easy integration with other technologies.

React will handle the user interface, allowing users to upload files, interact with encrypted content, and manage their account settings.

5.2. Backend – Flask (Python)

The backend of the platform will be built using **Flask**, a lightweight web framework for Python. Flask is known for its simplicity, flexibility, and ease of integration with various technologies, making it an excellent choice for small to medium-scale web applications. It will handle the business logic, file processing, and interactions with the database. The backend will provide essential APIs for:

- **User Authentication and Session Management:** Flask will manage user registration, login, and secure session handling.
- **File Upload and Encryption:** The backend will handle the encryption of uploaded files using the AES algorithm and store the encrypted files securely on the server.
- **Time Locking Mechanism:** Flask will manage the encryption keys and enforce the time-lock mechanism that ensures files cannot be decrypted until the specified time has passed.
- **Decryption Process:** Once the unlock time arrives, Flask will facilitate the decryption process for users.

Flask’s minimalist design and robust integration with other Python libraries (such as for AES encryption) make it a suitable choice for building the core backend functionalities of this platform.

6. System Architecture

6.1. High-Level Design

The high-level design of the system architecture outlines how the frontend, backend, database, and encrypted file storage interact. It describes the various components that make up the platform, including React.js for the frontend, Flask for the backend, and AES encryption for securing the files. The interactions between these components ensure the platform provides time-shifted encryption functionality with a secure user interface.

6.2. Flow Diagram

The flow diagram illustrates the sequence of actions performed by the user on the platform, including logging in, uploading a file, encrypting it, and performing decryption at a later time. The diagram provides a visual representation of how the platform’s features work together to provide time-based encryption and decryption.

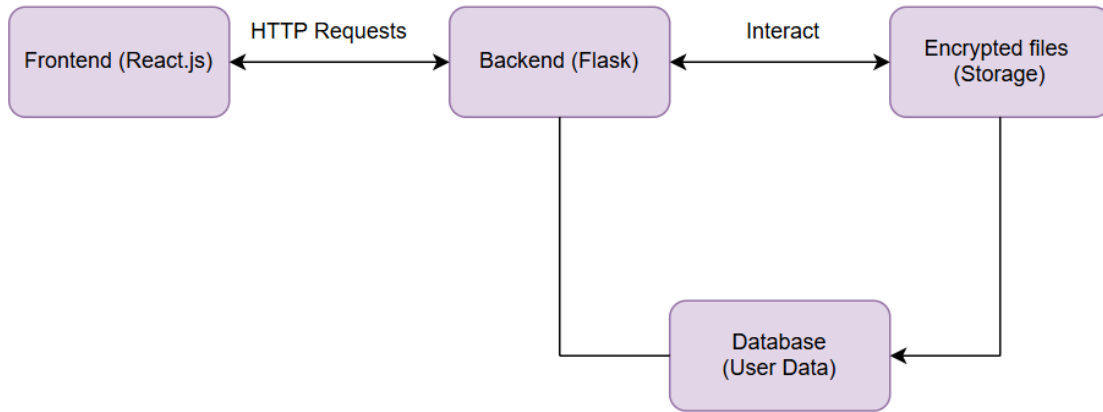


Figure 1: High-Level System Architecture

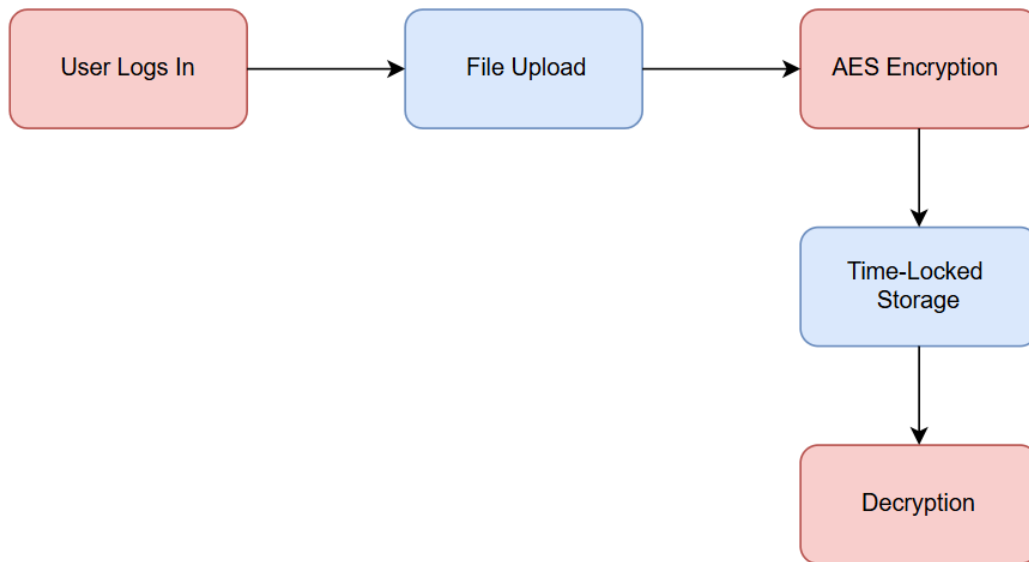


Figure 2: Flow Diagram of File Upload and Decryption Process

7. Working Methodology

The working methodology of the platform focuses on how users interact with the system to encrypt and decrypt their files securely, along with the time-lock mechanism that restricts access until a specified future time. This section explains the two critical components of the process: file upload with time input, and the AES encryption process.

7.1. File Upload and Time Input

The process begins with users logging into the platform and uploading their files (text, image, video, PDF, etc.). After uploading, users are prompted to specify a future time when the file should become accessible. This time input is crucial because it determines when the file can be decrypted by the recipient.

- **File Upload:**

- The user selects a file from their local storage.
- The file is then uploaded to the server, where it will be stored temporarily for encryption.

- **Time Input:**

- The user provides the date and time after which the file should be unlocked.
- This time is stored in the database alongside the encrypted file, ensuring that the decryption process is delayed until the specified time.

- **User Interface:**

- The UI includes a simple form where users can select the file for upload and specify the desired time (via a date and time picker).
- Once the file is uploaded and time is set, the platform proceeds to the encryption process.

7.2. AES Encryption Process

Once the file is uploaded and the time is set, the file undergoes AES encryption, ensuring that it remains inaccessible until the unlock time has passed.

- **Encryption Key Generation:**

- A unique encryption key is generated for the file using AES (Advanced Encryption Standard) encryption.
- The key is randomly generated for each file to ensure maximum security and uniqueness.

- **File Encryption:**

- The file is encrypted using the AES algorithm. This process involves transforming the file's content into a secure, unreadable format using the encryption key.
- AES encryption operates in blocks, ensuring that even large files are securely encrypted.

- **Storing Encrypted File:**

- After encryption, the file is saved in the server's encrypted file storage.
- The encryption key itself is stored securely in the database, ensuring it is not exposed until the unlock time has passed.

- **Time Locking Mechanism:**

- The time provided by the user is used to implement the time-lock feature.
- The decryption key is not accessible until the specified time has passed, ensuring that the file cannot be opened by the recipient prematurely.

- **User Feedback:**

- After the encryption process is completed, users are notified that their file has been successfully encrypted and is locked until the specified time.

8. Modules Implemented

The system is divided into various modules, each handling a specific functionality to provide seamless operation for the user. This section discusses two of the core modules of the platform: the File Upload Module and the Encryption Module.

8.1. File Upload Module

The File Upload Module allows users to upload their files to the server, where they will be processed and encrypted. This module ensures that the files are securely transferred and stored temporarily before encryption. It handles the following tasks:

- **User Interface for File Upload:**

- Provides a form where users can select a file from their local storage.
- Includes validation to ensure that only supported file formats (text, images, PDFs, videos, etc.) are uploaded.

- **File Transfer to Server:**

- After the user selects the file, it is sent to the backend via a secure HTTP request.
- The backend temporarily stores the file in a designated directory on the server.

- **Time Input:**

- Along with the file, the user is required to input the future time after which the file can be decrypted.
- This time input is sent along with the file for storage in the database.

- **Feedback to User:**

- Once the file is successfully uploaded, the system provides feedback to the user, confirming that the file has been received and stored for encryption.

- **Error Handling:**

- Handles any errors such as unsupported file types or upload failures and provides the user with relevant error messages.

8.2. Encryption Module

The Encryption Module is responsible for securing the uploaded files by applying AES encryption. This module ensures that the files remain confidential until the specified unlock time. It consists of the following components:

- **AES Key Generation:**
 - A unique AES encryption key is generated for each file.
 - This key is used to encrypt the file and is securely stored in the database.
- **File Encryption Process:**
 - The uploaded file is encrypted using the AES algorithm with the generated encryption key.
 - The file is transformed into an encrypted format that is unreadable until the decryption process is initiated after the unlock time.
- **Time-Locked Decryption Key Storage:**
 - The time input provided by the user determines when the encryption key will be made available for decryption.
 - The key is stored in a secure database with the associated time input.
- **Encrypted File Storage:**
 - After encryption, the file is stored in a secure encrypted storage location.
 - The file remains inaccessible to the recipient until the specified unlock time has passed.
- **Feedback to User:**
 - After successful encryption, the system provides feedback to the user confirming that the file has been encrypted and is now time-locked.
- **Error Handling:**
 - Handles errors that may occur during the encryption process, such as invalid file formats or issues with generating the encryption key.

9. AES Encryption Used in This Project

9.1. Why AES?

Advanced Encryption Standard (AES) is chosen for this project due to its strong security, speed, and wide adoption. AES is a symmetric block cipher that supports 128, 192, or 256-bit keys, operating on 128-bit blocks. It is widely used in securing sensitive data in government, finance, and private sectors.

AES provides:

- **High Security:** Resistant to known cryptographic attacks.

- **Fast Performance:** Efficient for encrypting large files.
- **Versatility:** Supports various data types including text, images, and videos.
- **Simplicity:** Easy to implement and integrate with web platforms.

9.2. Mode of Operation: AES-CBC (Cipher Block Chaining)

In this project, AES is used in Cipher Block Chaining (CBC) mode, where each plaintext block is XORed with the previous ciphertext block before being encrypted. The first block uses a random Initialization Vector (IV).

Encryption Formula

$$C_0 = E_K(P_0 \oplus IV)$$

$$C_i = E_K(P_i \oplus C_{i-1}), \quad \text{for } i > 0$$

Decryption Formula

$$P_i = D_K(C_i) \oplus C_{i-1}, \quad \text{for } i > 0$$

$$P_0 = D_K(C_0) \oplus IV$$

9.3. AES Encryption Rounds

AES performs multiple transformation rounds depending on key size. The process includes key expansion, substitution, permutation, and mixing of the data. The diagram below shows the high-level AES encryption round structure.

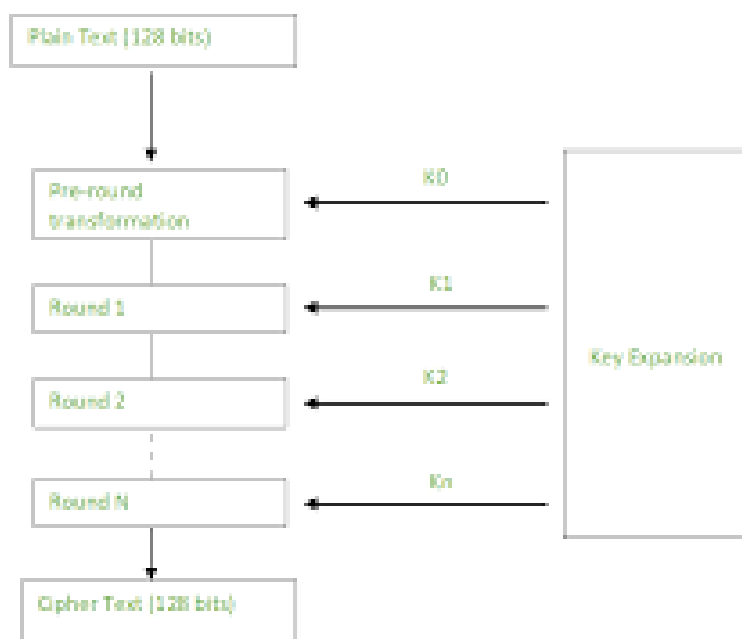


Figure 3: AES Encryption Round Structure

10. User Interface (Frontend)

10.1. Upload Page

This page allows users to select a file, choose a future date and time for decryption, and initiate encryption. Once encrypted, the user can download the encrypted file and share it via any platform.

10.2. Download & Decrypt Page

This page enables users to upload an encrypted file and decrypt it, provided the current time has reached or passed the specified unlock time. The platform checks the embedded time-lock before decryption.

11. Backend Workflow

11.1. Flask API Endpoints

The backend is built using Flask, a lightweight Python web framework. The key API endpoints include:

- **/upload:** Accepts files for encryption along with the time-lock timestamp. Returns the encrypted file for download.
- **/decrypt:** Accepts an encrypted file and checks the current time against the time-lock before performing decryption.

11.2. File Handling

Uploaded files are temporarily stored on the server during encryption/decryption. AES encryption is applied to:

- Read file bytes from input.
- Apply AES-CBC encryption using a key and IV.
- Save encrypted output with metadata (e.g., unlock timestamp).
- On decryption, validate the unlock time before decrypting and sending back the original content.

12. Time Lock Mechanism

12.1. Time Comparison Logic

The time-lock feature ensures that encrypted content can only be decrypted after a specified future timestamp. The logic includes:

- Capturing the current server time using Python's `datetime` module.

- Comparing it against the user-defined unlock time.
- Proceeding with decryption only if the current time is equal to or later than the unlock time.

12.2. Preventing Early Access

To prevent users from bypassing the time lock:

- All time comparisons are handled on the backend server.
- The encrypted file format does not reveal the decryption key or plaintext before the unlock time.
- Attempts to decrypt before the allowed time will result in an error message or denial of access.

13. Security Considerations

13.1. Confidentiality

Confidentiality ensures that only authorized users can access the content. In this project:

- AES encryption with a strong key ensures that the data remains unreadable to unauthorized parties.
- The encryption key is never exposed to the client; it remains securely handled on the server side.
- Encrypted files are stored in a secure directory with restricted access.

13.2. Integrity

Integrity guarantees that the content has not been tampered with. Measures include:

- Hash functions (e.g., SHA-256) can be used to generate a checksum of the file before encryption.
- Upon decryption, the checksum is recomputed and compared with the original to verify file integrity.
- Any mismatch in checksums indicates potential tampering or corruption.

14. Testing and Results

14.1. Functional Testing

The platform was tested for core functionalities, including:

- File upload and AES encryption with a future time lock.

- Backend storage of encrypted files.
- Prevention of early decryption before the allowed time.
- Successful file decryption after the time threshold is reached.
- User feedback and error handling on the frontend.

All functionalities were tested manually using different file types (text, image, PDF), and the system responded as expected in each case.

14.2. Sample Screenshots

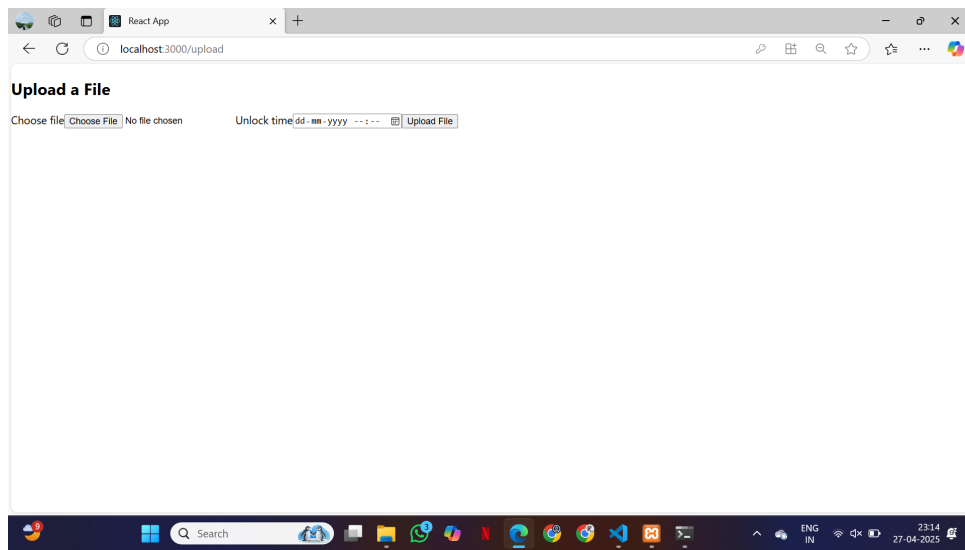


Figure 4: Upload Page Interface

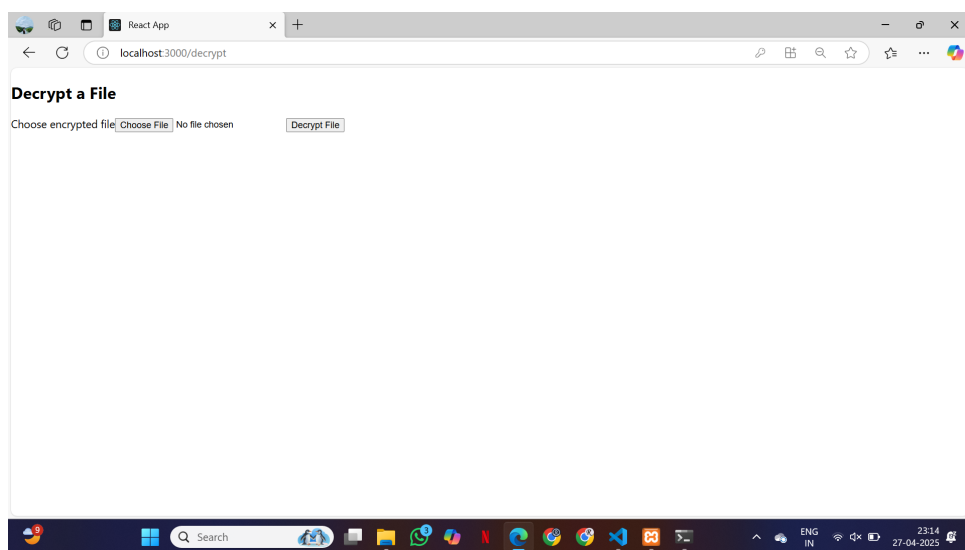


Figure 5: Decrypt Page Interface

15. Limitations

15.1. Only Text File Supported

Currently, the platform supports encryption and decryption of text files only. Although the AES algorithm can handle binary data (like images, PDFs, and videos), the current implementation is limited due to file handling logic and storage format constraints.

15.2. Frontend Upload Errors

Occasionally, users may encounter errors while uploading large files or using unsupported formats. These issues are mainly due to limited frontend validation and browser compatibility. Proper error messages are shown, but further refinement is needed for a smoother user experience.

16. Conclusion

16.1. Summary

This project successfully demonstrates the concept and implementation of a Time-Shifted Encryption Web Platform using AES encryption in CBC mode. The platform enables users to securely upload encrypted files and ensures that decryption is only possible after a specified future time, enhancing data confidentiality and control.

16.2. Learnings

Throughout the development of this project, several key technical skills were gained, including:

- Integration of frontend (React) and backend (Flask) components.
- Practical implementation of AES encryption and time-based logic.
- File handling and secure communication between client and server.
- Designing user-friendly interfaces and handling common errors.

The project provided hands-on experience with web security principles, real-time development, and encryption mechanisms.

17. Future Work

17.1. Support for PDF, Image, and Video

Currently, the platform supports only text file encryption and decryption. Future enhancements will include the ability to upload, encrypt, and decrypt other file types such as PDFs, images (e.g., PNG, JPEG), and video files. This will make the platform more versatile and practical for real-world use cases.

17.2. Decryption Link Sharing

An important feature to be added is the generation of a secure decryption link that can be shared with recipients. This link will contain metadata like file ID and timestamp, allowing authorized users to access and decrypt the file only after the preset time. This improves usability while maintaining the time-lock security principle.

18. References

1. National Institute of Standards and Technology (NIST). “FIPS PUB 197: Advanced Encryption Standard (AES)”, 2001. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
2. William Stallings, “Cryptography and Network Security: Principles and Practice”, Pearson Education, 7th Edition.
3. Flask Documentation. <https://flask.palletsprojects.com/>
4. React Documentation. <https://reactjs.org/docs/getting-started.html>

19. Appendix

19.1. Code Snippets

Upload and Encrypt (Flask Backend)

```
@app.route('/upload', methods=['POST'])
def upload_file():
    file = request.files['file']
    key = get_random_bytes(16)
    iv = get_random_bytes(16)
    ciphertext = encrypt_aes(file.read(), key, iv)
    # Save the encrypted file and return download link
```

Time Lock Validation

```
@app.route('/download/<file_id>', methods=['GET'])
def download_file(file_id):
    if datetime.now() >= stored_unlock_time:
        decrypted = decrypt_aes(ciphertext, key, iv)
        return send_file(BytesIO(decrypted), download_name='file.txt')
    else:
        return jsonify({"error": "Access denied: Too early!"})
```